

Relatorio tecnico - CloudTask AI SaaS

Registro das entregas realizadas nas Aulas 1 a 4

Campo	Valor
Projeto	CloudTask AI SaaS
Disciplina	Computacao em Nuvem
Etapa	Aulas 1, 2, 3 e 4
Branch de trabalho	aulas-03-04-postgres-config
Data	16/06/2026

Este documento consolida o que foi implementado ate aqui no projeto. O foco da etapa foi sair de uma API minima em FastAPI, padronizar a execucao com Docker, adicionar persistencia com PostgreSQL e preparar a aplicacao para configuracao por ambiente e execucao futura em nuvem.

1. Resumo executivo

A aplicacao esta funcional localmente via Docker Compose. Ela possui endpoints basicos de saude, documentacao Swagger, banco PostgreSQL, CRUD de tarefas e readiness check separado para validar o banco.

Aula	Tema	Status
1	FastAPI minimo	Concluida
2	Docker e Docker Compose	Concluida
3	PostgreSQL e CRUD de tarefas	Concluida
4	Configuracao, seguranca e readiness	Concluida

2. Entregas por aula

Aula 1 - API minima com FastAPI

- Criacao da estrutura inicial da aplicacao em app/.
- Instancia principal do FastAPI em app/main.py.
- Endpoint GET / com metadados do projeto.
- Endpoint GET /health para liveness simples.
- Swagger automatico disponivel em /docs.
- Schemas Pydantic iniciais em app/schemas.py.

Aula 2 - Docker e Docker Compose

- Dockerfile multi-target para desenvolvimento e producao.
- Servico api no docker-compose.yml.
- Mapeamento da porta 8000 para acesso local.
- Volume de codigo para hot reload em desenvolvimento.
- README com comandos para subir, parar e inspecionar a aplicacao.

Aula 3 - PostgreSQL e CRUD de tarefas

- Servico db com PostgreSQL 16 no Docker Compose.
- Volume pgdata para persistencia dos dados.
- Dependencias sqlalchemy e psycopg2-binary adicionadas.
- Camada de banco em app/db/database.py.
- Modelo Task em app/db/models.py.
- Schemas TaskCreate, TaskUpdate e TaskRead em app/db/schemas.py.
- CRUD completo em app/api/routes_tasks.py.

Aula 4 - Configuracao, seguranca e readiness

- Criado app/core/config.py com pydantic-settings.
- DATABASE_URL, APP_ENV, SECRET_KEY e demais variaveis passaram a vir do ambiente.
- Criado GET /health/ready com SELECT 1 no PostgreSQL.
- Mantido GET /health como liveness puro, sem consulta ao banco.
- Adicionado TrustedHostMiddleware usando TRUSTED_HOSTS.
- Adicionado HSTS condicional para ambientes fora de desenvolvimento.
- Uvicorn configurado com --proxy-headers para preparar uso atras de proxy/ALB.
- Documentos conceituais de VPC/IAM/seguranca ja presentes em docs/conceitos/.

3. Arquitetura atual

Nesta etapa a arquitetura ainda e local, mas ja simula uma separacao importante para nuvem: a API roda em um container separado do banco. A API le configuracoes do ambiente e conversa com o PostgreSQL pelo nome do servico db dentro da rede do Docker Compose.

Navegador/Swagger -> localhost:8000 -> cloudtask-api -> db:5432 -> cloudtask-db

4. Endpoints disponiveis

Metodo	Caminho	Finalidade
GET	/	Metadados da aplicacao
GET	/health	Liveness probe: processo HTTP vivo
GET	/health/ready	Readiness probe: valida PostgreSQL
POST	/tasks	Criar tarefa
GET	/tasks	Listar tarefas
GET	/tasks/{task_id}	Buscar tarefa por ID
PUT	/tasks/{task_id}	Atualizar tarefa
DELETE	/tasks/{task_id}	Remover tarefa

5. Principais arquivos do projeto

Arquivo	Papel
app/main.py	Entrada FastAPI, middlewares e registro de rotas
app/core/config.py	Configuracao central via .env
app/api/routes_health.py	Liveness e readiness
app/api/routes_tasks.py	CRUD de tarefas
app/db/database.py	Engine e sessao SQLAlchemy
app/db/models.py	Modelo Task
app/db/schemas.py	Schemas Pydantic de tarefas
docker-compose.yml	API e PostgreSQL local
.env.example	Modelo de variaveis de ambiente
docs/conceitos/aws-networking.md	Conceitos de rede AWS
docs/conceitos/security-model.md	Conceitos de seguranca, IAM e LGPD

6. Como executar e validar

Subir os containers:

```
cd D:\Estudos\Workspace\ComputacaoNuvem_MicroSaas
docker compose up --build -d
```

Acessar a documentacao interativa:

```
http://localhost:8000/docs
```

Comandos uteis de verificacao:

```
docker compose ps
docker compose logs -f api
docker compose exec db psql -U cloudtask -d cloudtask
SELECT * FROM tasks;
```

7. Pendencias e proximos passos

- Realizar commit da branch aulas-03-04-postgres-config, se ainda nao foi feito.
- Executar smoke test final pelo Swagger apos cada rebuild.
- Iniciar Aula 5: uploads com fallback local e Amazon S3.
- Manter o .env real fora do versionamento.
- Ao avancar para AWS, substituir credenciais locais por IAM Roles ou Secrets Manager.

8. Conclusao

As quatro primeiras aulas deixam o projeto com uma base solida para nuvem: API documentada, execucao padronizada por container, persistencia relacional e configuracao desacoplada do codigo. A partir daqui, o caminho natural e adicionar armazenamento de arquivos com S3 e preparar a aplicacao para orquestracao em Kubernetes.